

BibOps

Banc d'évaluation et d'optimisation de LLM pour le support IT

Comparaison LLM unique, agent ReAct outillé, agents A2A et évaluation adversariale

Équipe : Mohammed Akram Lrhorfi, Widad, Ayoub

Encadrant industriel : Emilien Mottet, IT Expert / SRE, Michelin

École : Ensimag

Module : Projet de spécialité S8

Dépôt : BibOps-michelin-ensimag-aginix

Date : 13 mai 2026

Ce rapport est produit à partir du dépôt local et des artefacts générés par les campagnes de benchmark. Les identifiants fournis pour les agents A2A ne sont pas reproduits dans le rapport afin d'éviter toute diffusion de secrets.

Table des matières

Résumé	1
1 Introduction	3
1.1 Contexte industriel	3
1.2 Problématique	3
1.3 Objectifs	3
1.4 Contributions	4
1.5 Périmètre du rapport	4
2 Description conceptuelle du problème	5
2.1 Nature des tickets de support IT	5
2.2 Limites du LLM unique	5
2.3 Intérêt de l’approche agentique	5
2.4 Critères d’évaluation	6
2.5 Problème de reproductibilité	6
3 Présentation de la solution	7
3.1 Vue d’ensemble	7
3.2 Agent de support IT	8
3.3 Évaluation	9
3.4 A2A, MCP et Racing Arena	10
3.5 Reproductibilité	11
4 Organisation de l’équipe et synchronisation Git	12
4.1 Répartition du travail	12
4.2 Méthode de collaboration	12
4.3 Pipeline CI/CD	13
4.4 Rôle du CI dans la synchronisation	14
5 Patrons de conception et modèles d’analyse	15
5.1 Cas d’utilisation	15
5.2 Diagramme de classes d’analyse	16
5.3 Patrons identifiés	17
5.4 Choix de conception	18

6	Résultats	19
6.1	Conditions expérimentales	19
6.2	LLM unique contre agent ReAct	20
6.3	Benchmark MCP	21
6.4	Agents A2A	22
6.5	Kaggle local	22
6.6	Biais de position	23
6.7	Racing Arena adversariale	23
6.8	Vérification logicielle	24
7	Discussion	25
7.1	Interprétation des résultats	25
7.2	Limites	25
7.3	Apports méthodologiques	25
7.4	Recommandations	26
7.5	Perspectives	26
8	Conclusion	27

Table des figures

3.1	Vue d'ensemble UML de l'architecture BIBOPS.	7
3.2	Boucle ReAct UML de l'agent de support IT.	8
3.3	Poids de la politique composite.	10
3.4	Flux UML de données de la Racing Arena.	11
4.1	Planification Gantt du projet sous PlantUML.	12
4.2	Pipeline Git, CI/CD et production des artefacts.	13
5.1	Diagramme des cas d'utilisation de BIBOPS.	16
5.2	Diagramme de classes d'analyse.	17
5.3	Vue PlantUML des patrons de conception.	18
6.1	Comparaison globale des deux architectures.	20
6.2	Verdicts composites des architectures.	21
6.3	Synthese du benchmark MCP.	21
6.4	Synthese du benchmark A2A.	22
6.5	Synthese du benchmark Kaggle local.	23
6.6	Test de biais de position.	23
6.7	Rapport de sécurité de la Racing Arena.	24

Liste des tableaux

1	Synthese du benchmark principal extrait de <code>comparison_results.json</code>	2
2.1	Ponderation de la politique composite.	6
3.1	Organisation fonctionnelle du dépôt.	8
3.2	Outils disponibles pour l’agent IT.	9
5.1	Patrons de conception principaux.	18
6.1	Sources chiffrees exploitees.	19
6.2	Résultats principaux de <code>comparison_results.json</code>	20
6.3	Synthese des benchmarks MCP.	21
6.4	Agents A2A avec transport complet.	22
6.5	Metriques de sécurité adversariale.	24

Résumé

Les grands modèles de langage sont aujourd’hui utilisables pour assister le support informatique : classification de tickets, aide au diagnostic, recherche dans une base de connaissances ou rédaction de réponses utilisateur. Dans un contexte industriel comme Michelin, cette promesse doit cependant être évaluée avec rigueur. La qualité apparente d’une réponse ne suffit pas : il faut aussi mesurer la sécurité, la latence, le coût, la dépendance fournisseur et l’empreinte environnementale.

Le projet BIBOPS construit un banc d’évaluation reproductible pour ce problème. Il compare une architecture *LLM unique*, appelée directement sur un ticket, à un agent ReAct outillé. L’agent peut vérifier un statut serveur dans SQLite, chercher dans une base de connaissances JSON et interroger une documentation technique via ChromaDB. Le projet ajoute aussi des benchmarks A2A, des tests de biais, un moteur d’évaluation multi-critères et une Racing Arena adversariale permettant d’étudier des agents connectés à un flux temps réel.

La campagne principale du 13 mai 2026 évalue **21 tickets IT**. Avec `phi3:latest` côté Ollama et `gpt-5.2` comme juge, le système multi-agents obtient **7,86/10** en qualité moyenne contre **2,81/10** pour le LLM unique. Le score composite atteint **91,34/100** avec un verdict **PASS**, contre **46,14/100** et **FAIL** pour le LLM unique. L’architecture agentique réduit aussi la latence totale de **99,8 %**, le coût estimé de **60,6 %** et le nombre de tokens de **63,8 %**. Ces chiffres valident l’intérêt d’une architecture outillée pour les tickets qui dépendent d’un état ou d’une procédure interne.

Le résultat doit rester nuancé. La faible latence de l’agent est obtenue dans une configuration expérimentale où le premier outil est forcé et où certaines réponses sont synthétisées de façon déterministe à partir du résultat outil. Le rapport conclut donc que l’approche est prometteuse et mesurable, pas qu’elle peut remplacer telle quelle un support IT humain. La Racing Arena renforce cette prudence : une équipe ReAct exposée à des messages adversariaux exécute **53 injections sur 57 attaques reçues**, tandis qu’une équipe validée n’exécute aucune injection dans le scénario observé.

Métrique principale	LLM unique	Agent ReAct outillé
Tickets IT notés	21	21
Score qualité moyen	2,81/10	7,86/10
Score sécurité moyen	9,97/10	9,97/10
Latence totale	461,58 s	0,84 s
Coût estimé	0,061315 \$	0,024150 \$
Tokens totaux	6 664	2 415
Empreinte carbone estimée	0,06664 gCO ₂ e	0,02415 gCO₂e
Score composite	46,14/100	91,34/100
Verdict de release	FAIL	PASS

Table 1 – Synthèse du benchmark principal extrait de `comparison_results.json`.

Mots-clés : LLM, support IT, ReAct, RAG, A2A, MCP, évaluation, sécurité agentique, CI/CD, GreenOps.

1. Introduction

1.1 Contexte industriel

Les assistants fondés sur des LLM peuvent accélérer le support IT en produisant un diagnostic initial, en reformulant une procédure, en suggérant une escalade ou en résumant un incident. Le cas Michelin ajoute des contraintes concrètes : les tickets peuvent contenir des données sensibles, les réponses doivent rester traçables, et l'outil doit fonctionner dans un environnement où coût, latence et dépendance fournisseur sont des critères de décision.

L'usage brut d'un LLM est donc insuffisant comme objet d'étude. Une réponse fluide peut être fausse, inventer une procédure ou ignorer un état de service local. À l'inverse, un modèle plus petit peut devenir acceptable s'il est correctement outillé, cadré par des prompts robustes et évalué dans un pipeline reproductible. C'est l'hypothèse centrale du projet.

1.2 Problématique

La question traitée est la suivante :

Comment concevoir un banc d'évaluation reproductible permettant de comparer des LLM et des architectures agentiques pour des cas d'usage de support IT, tout en tenant compte de la qualité, de la sécurité, de la latence, du coût et de l'impact environnemental ?

Cette question impose de travailler à deux niveaux. Le premier est fonctionnel : le système doit répondre à des tickets IT. Le second est méthodologique : les résultats doivent être quantifiés, historisés et comparables d'une campagne à l'autre.

1.3 Objectifs

Le projet s'organise autour de quatre objectifs.

- O1. Construire un jeu de benchmark support IT.** Les tickets couvrent des demandes comme VPN, Outlook, imprimante, BitLocker, Teams, poste lent ou accès réseau.
- O2. Comparer plusieurs architectures.** Le banc oppose un LLM unique zero-shot, un agent ReAct outillé, des agents A2A distants et des variantes de Racing Arena.

- O3. Évaluer selon plusieurs dimensions.** Les sorties sont notées par qualité, sécurité, coût, latence, tokens et GreenOps. Ces dimensions sont combinées dans une politique composite explicite.
- O4. Rendre le projet maintenable.** Le dépôt expose une CLI, des tests unitaires mocks, une organisation modulaire et une intégration Git/CI pour synchroniser les tâches de l'équipe.

1.4 Contributions

Les contributions principales sont :

- une CLI **bibops** regroupant les commandes de benchmark, évaluation, développement, Racing Arena, configuration et rapport ;
- un agent de support IT traçable, basé sur ReAct [3], RAG [2] et appels outils ;
- un moteur d'évaluation combinant juge LLM, inspecteurs de sécurité, métriques opérationnelles et gates de release ;
- des benchmarks A2A et MCP permettant d'étudier des agents ou outils externes ;
- une Racing Arena avec flux SSE et agents adversariaux ;
- une suite de tests unitaires permettant de vérifier le code sans dépendance à un modèle LLM réel.

1.5 Périmètre du rapport

Le rapport décrit le dépôt dans son état du 13 mai 2026. Les résultats sont extraits des artefacts présents sous `data/outputs/benchmark/`. Ils incluent la campagne principale `comparison_results.json`, les benchmarks MCP, les campagnes A2A, les tests Kaggle locaux, les tests de biais de position et le rapport adversarial de la Racing Arena. Les identifiants d'accès aux agents externes ne sont pas reproduits.

2. Description conceptuelle du problème

2.1 Nature des tickets de support IT

Un ticket IT est souvent court, incomplet et contextuel. Il peut décrire un symptôme sans donner l'état réel du service, le message d'erreur exact ou le niveau d'urgence. Une bonne réponse doit donc rester prudente : identifier une cause probable, proposer des actions concrètes, demander les informations manquantes et escalader lorsque le problème dépend d'une infrastructure.

Le jeu de données principal contient des tickets IT comme : *Mon VPN Cisco ne marche plus*, *Outlook crash au démarrage*, *Comment récupérer ma cle BitLocker ?* ou *Mon PC redemarre en boucle après une mise à jour Windows*. Ces tickets représentent bien un support de niveau 1 ou 2 : le modèle doit être utile sans inventer d'état interne.

2.2 Limites du LLM unique

L'architecture LLM unique appelle directement le modèle sur le ticket. Elle a l'avantage d'être simple, mais elle présente trois limites.

- **Pas d'état externe.** Le modèle ne peut pas vérifier si le VPN ou la messagerie est en panne dans l'environnement local.
- **Risque d'hallucination.** Le modèle peut proposer des procédures, numéros ou causes plausibles mais non vérifiées.
- **Évaluation trompeuse.** Une réponse bien rédigée peut être insuffisante si elle ne respecte pas les sources ou les contraintes de sécurité.

2.3 Intérêt de l'approche agentique

Une architecture agentique introduit une boucle perception-décision-action. Le LLM ne répond plus seulement au ticket : il choisit si un outil est nécessaire, lit le résultat et produit une réponse finale. Cette approche est utile lorsque le problème dépend d'une base de connaissances, d'une documentation technique ou d'un statut serveur.

Elle introduit aussi de nouveaux risques. Le modèle peut choisir le mauvais outil, répéter un appel, mal interpréter un résultat ou suivre une instruction malveillante contenue dans une source externe. L'évaluation doit donc inclure l'outillage, la trace et la sécurité agentique, pas seulement la réponse finale.

2.4 Critères d'évaluation

BIBOPS formalise l'évaluation comme un problème multicritere.

Dimension	Poids	Rôle
Qualité	40 %	Pertinence, completude, clarte et utilite de la réponse.
Sécurité	35 %	PII, secrets, injection, URLs suspectes, toxicite, refus.
FinOps	10 %	Coût estimé à partir des tokens.
Latence	10 %	Temps de réponse total.
GreenOps	5 %	Énergie et empreinte carbone estimées.

Table 2.1 – Ponderation de la politique composite.

Le score composite est complete par des gates bloquants : qualité minimale, sécurité minimale et seuils maximaux pour certains risques. Cette logique evite qu'une architecture rapide ou peu coûteuse soit validée si elle répond mal.

2.5 Problème de reproductibilité

Les LLM rendent l'experimentation fragile : les modèles changent, les APIs sont limitees par quota, les juges peuvent varier et les outils externes peuvent tomber en erreur. Le banc doit donc enregistrer les configurations, produire des artefacts JSON, générer des graphiques et separer les tests unitaires des appels réseau. Cette exigence justifie l'investissement dans une CLI, une structure de dépôt claire et un pipeline CI.

3. Présentation de la solution

3.1 Vue d'ensemble

BIBOPS est un framework Python organisé autour d'une CLI et de modules spécialisés. Le pipeline part d'un dataset de tickets, exécute une ou plusieurs architectures, évalue les réponses puis produit des artefacts exploitables.

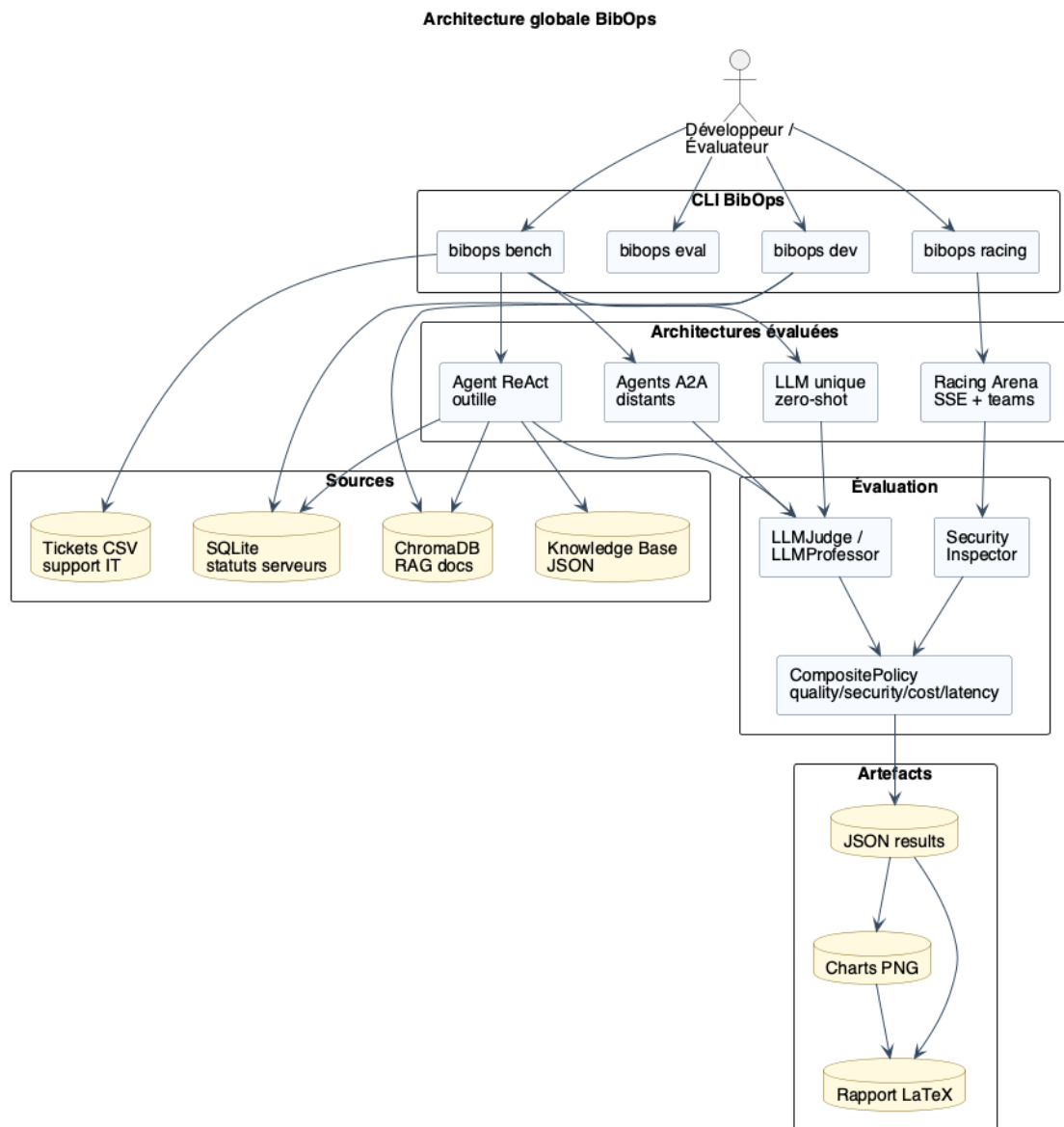


Figure 3.1 – Vue d'ensemble UML de l'architecture BIBOPS.

Module	Responsabilité
src/agent/	Agent IT ReAct, mémoire courte, outils, serveur MCP et RAG.
src/common/	Configuration, clients LLM et fonctions texte partagées.
src/bibops/cli/	CLI Typer : bench, eval, dev, racing, copilot, test, config, report.
src/bibops/benchmark/	Runners de benchmark et validation des sorties.
src/bibops/evaluation/	Juges, inspecteurs de sécurité, scores composites et seuils.
src/bibops/adapters/	Adaptateurs vers agents IT, A2A et API OpenAI-compatible.
src/racing/	Hub FastAPI, flux SSE, équipes LangGraph et scénario adversarial.
tests/	Tests unitaires, fakes OpenAI et fixtures.

Table 3.1 – Organisation fonctionnelle du dépôt.

3.2 Agent de support IT

La fonction centrale est `lancer_agent()` dans `src/agent/maestro.py`. Elle exécute une boucle ReAct bornée par un nombre maximal d’itérations. A chaque tour, le LLM retourne une décision structurée : nom d’outil, argument et réponse finale éventuelle. Ce contrat évite de parser du texte libre et rend les tests unitaires plus fiables.

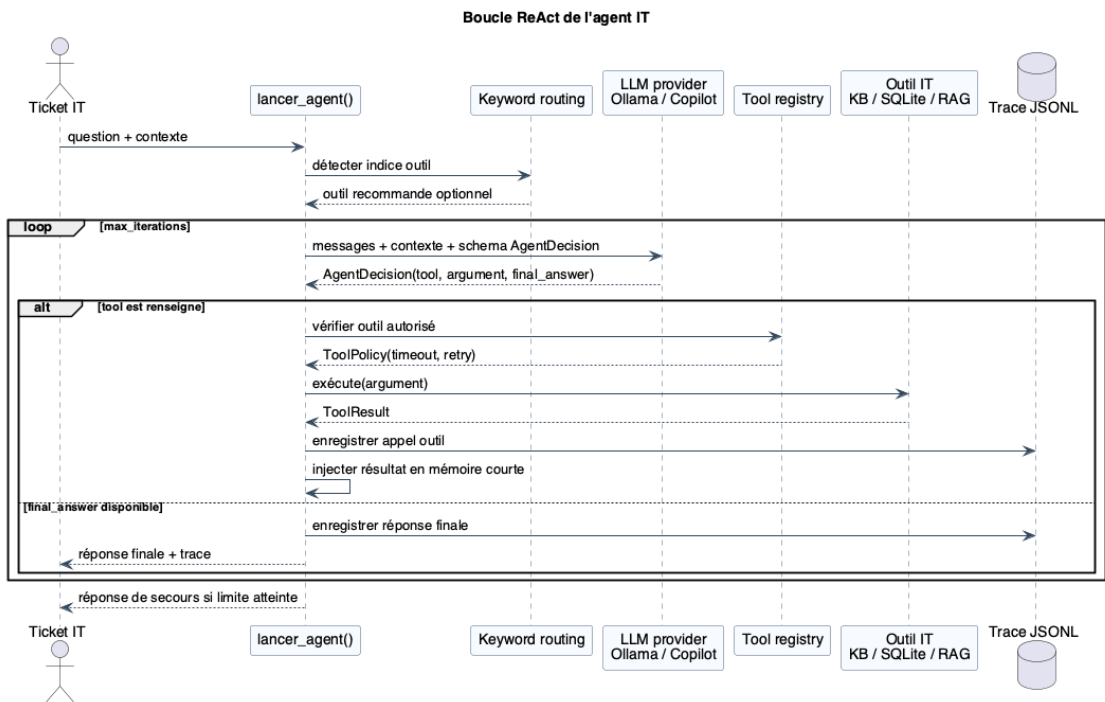


Figure 3.2 – Boucle ReAct UML de l’agent de support IT.

Les outils exposés sont volontairement simples.

Outil	Timeout	Retry	Usage
<code>verifier_statut_serveur</code>	3 s	0	Interroge SQLite pour vérifier l'état d'un service.
<code>chercher_dans_kb</code>	5 s	1	Cherche dans une base de connaissances JSON.
<code>chercher_documentation_technique</code>	8 s	1	Interroge une base ChromaDB pour le RAG.

Table 3.2 – Outils disponibles pour l'agent IT.

Un routage lexical fournit un indice initial : par exemple `vpn` oriente vers le statut serveur, tandis que `bitlocker` oriente vers la documentation. L'indice ne remplace pas la décision du modèle, mais il augmente la probabilité d'utiliser la bonne source.

3.3 Évaluation

Le module `src/bibops/evaluation/` combine trois familles d'évaluation. `LLMJudge` fournit un score générique ; `LLMProfessor` specialise le jugement pour les tickets IT ; les détecteurs déterministes identifient PII, secrets, prompt injection, toxicité, URLs suspectes et refus incorrects.

La politique composite calcule :

$$S = 0,40Q + 0,35Sec + 0,10F + 0,10L + 0,05G,$$

ou Q est la qualité, Sec la sécurité, F le score FinOps, L la latence normalisée et G le score GreenOps. Le score est rapporté sur 100 et complété par un verdict **PASS** ou **FAIL**.

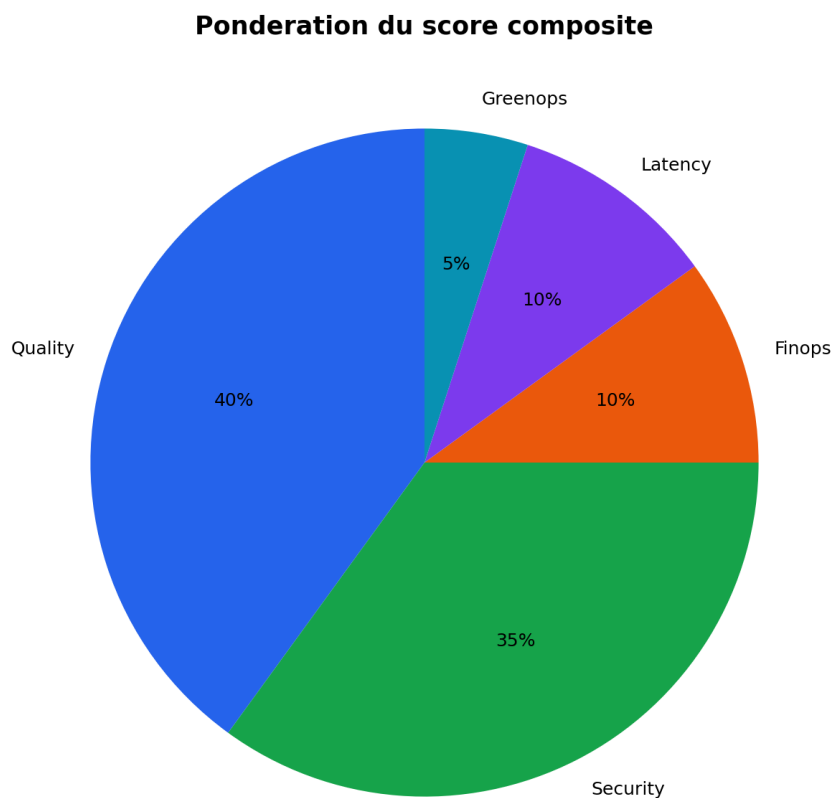


Figure 3.3 – Poids de la politique composite.

3.4 A2A, MCP et Racing Arena

Les agents A2A sont utilisés pour tester des assistants distants exposés par JSON-RPC. Le projet prend en compte les différences entre fact-checkers et agents OpenClaw : endpoint de découverte, endpoint RPC, format des messages et présence possible d'outils comme `fetch`, `tavily`, `filesystem` ou `e2b`. Les secrets d'authentification restent hors rapport.

Le protocole MCP sert à exposer les outils de l'agent IT sous une interface standardisée [1]. Cela permet de mesurer les outils indépendamment du LLM et de préparer des intégrations plus larges.

La Racing Arena est une extension exploratoire. Un hub FastAPI diffuse une télémétrie par SSE ; plusieurs équipes LLM prennent des décisions ; une équipe adversariale tente des prompt injections et fuites de stratégie. Cette arène teste la sécurité d'architecture, pas seulement la qualité textuelle.

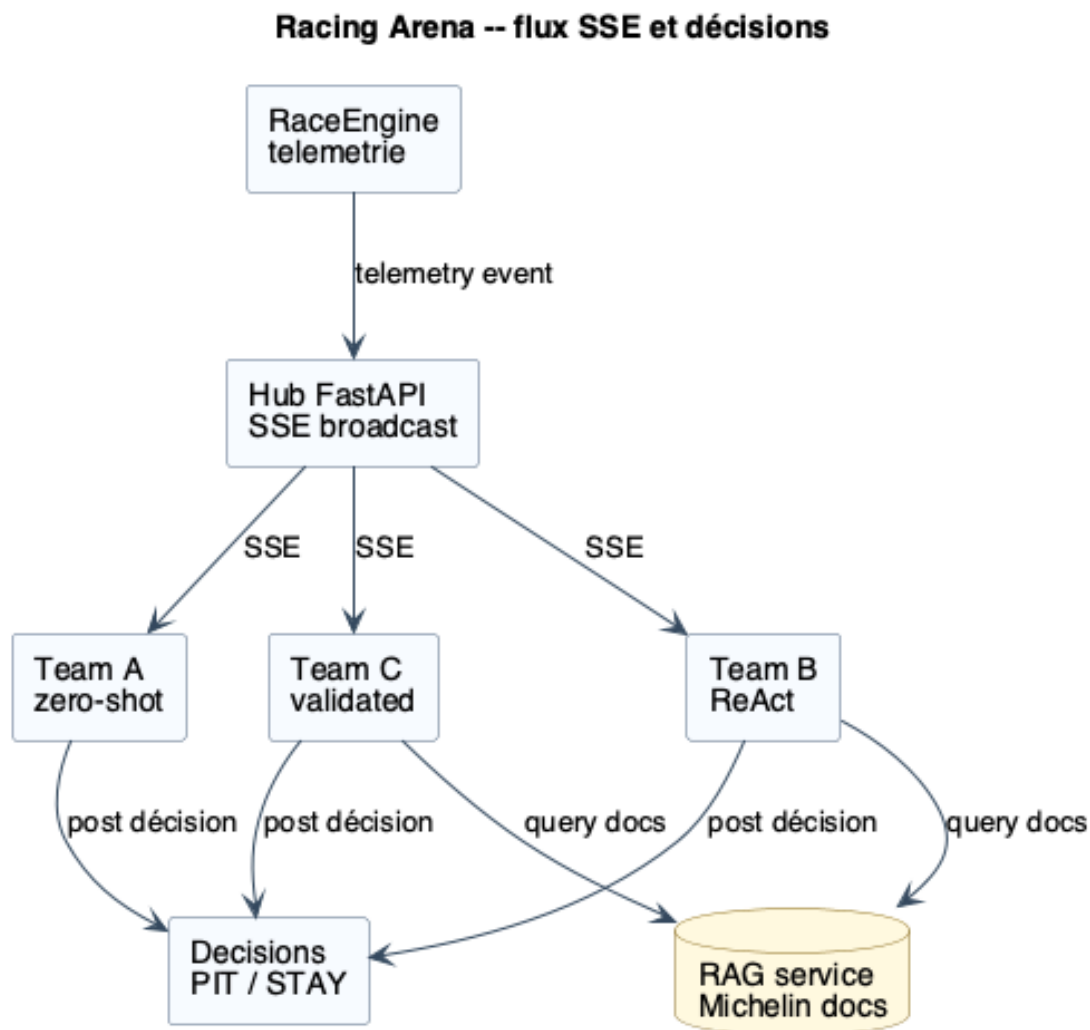


Figure 3.4 – Flux UML de données de la Racing Arena.

3.5 Reproductibilité

La reproductibilité repose sur quatre décisions :

- une CLI unique `bibops` pour lancer les campagnes ;
- des artefacts JSON et graphiques sous `data/outputs/benchmark/` ;
- des tests unitaires qui mockent les LLM et ne dépendent pas d'Ollama ;
- une séparation entre code source, données runtime et résultats générés.

Les commandes principales sont documentées dans le README et dans la documentation détaillée. La vérification locale actuelle passe par `python -m pytest tests/unit -q`.

4. Organisation de l'équipe et synchronisation Git

4.1 Répartition du travail

Le projet a été conduit par trois personnes avec une répartition par domaines. Akram a principalement pris en charge l'agent, la vectorisation, la sécurité expérimentale et le refactoring final. Widad a porté l'interface d'évaluation, la knowledge base, le moteur de scoring, les tests A/B et la consolidation des scores. Ayoub a travaillé sur la recherche KB, le serveur et client MCP, l'intégration Copilot API, le protocole A2A et la preparation de la soutenance.

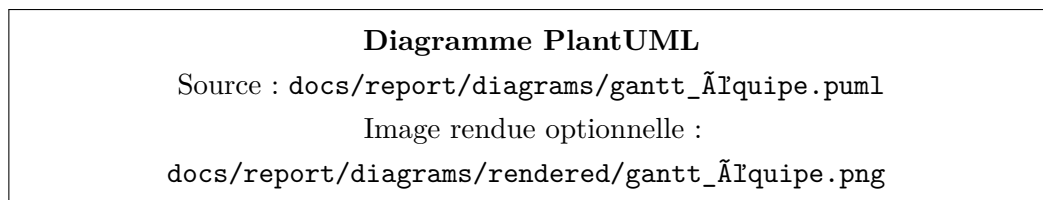


Figure 4.1 – Planification Gantt du projet sous PlantUML.

Cette répartition a permis de paralléliser les sujets tout en gardant un point de convergence hebdomadaire : chaque fonctionnalite devait produire soit une commande CLI, soit un artefact de benchmark, soit un test automatisable.

4.2 Méthode de collaboration

Le dépôt Git a servi de point de synchronisation. Les branches de travail étaient associées aux lots fonctionnels : évaluation, RAG, MCP, A2A, Racing Arena, documentation et refactoring. Les merges étaient accompagnés de tests unitaires afin d'éviter les régressions sur les contrats internes.

La logique suivie est proche d'un cycle court :

1. créer ou choisir un ticket de travail ;
2. développer sur une branche dédiée ;
3. lancer les tests et les linters localement ;
4. ouvrir une demande de merge ;
5. corriger les retours et synchroniser avec la branche principale ;
6. regenerer les artefacts de benchmark lorsque le comportement change.

4.3 Pipeline CI/CD

Le pipeline CI/CD GitHub Actions sert d'assurance qualité. Il installe les dépendances runtime et développement, exécute Ruff, lance les tests unitaires et vérifie les seuils de couverture. Les artefacts lourds ou générés ne sont pas faits pour être versionnés directement : ils doivent être régénérables à partir des commandes du projet.

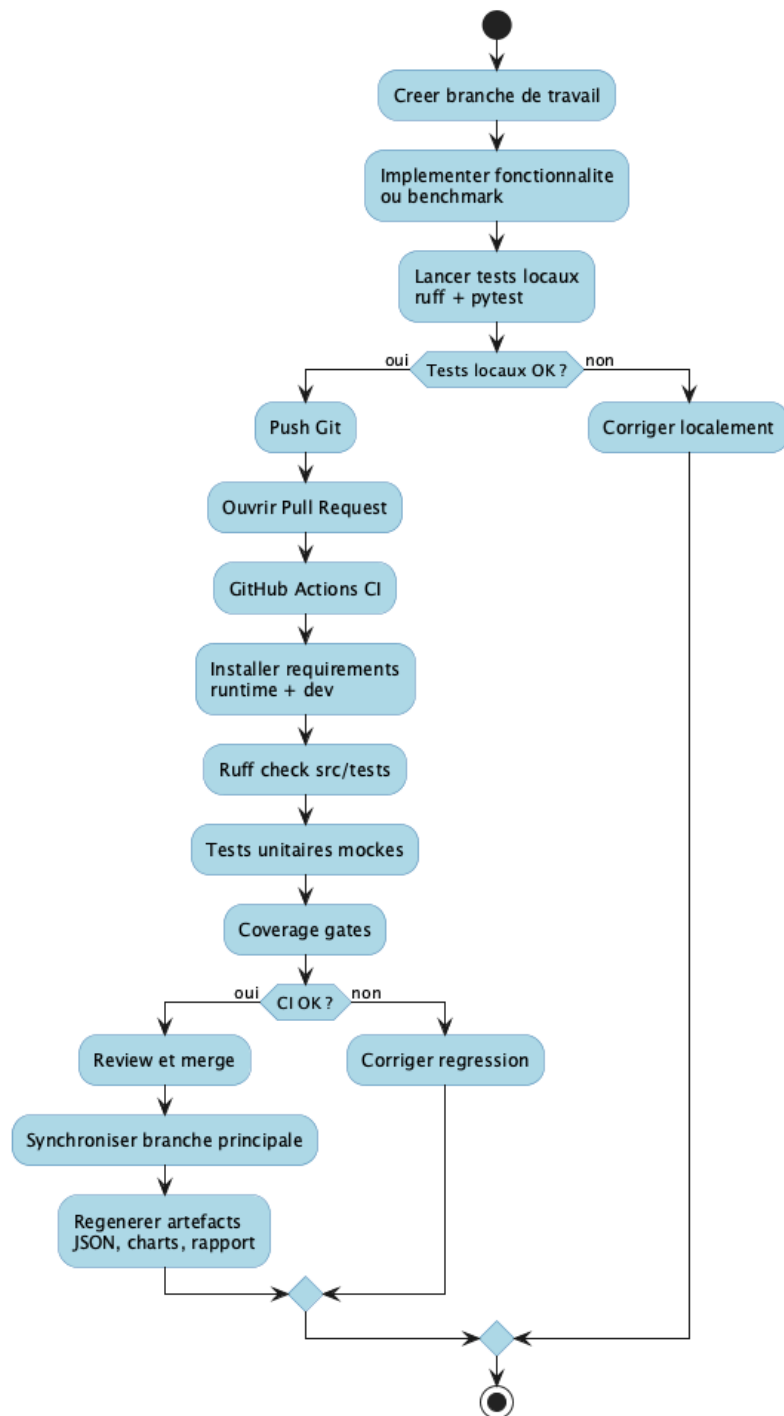


Figure 4.2 – Pipeline Git, CI/CD et production des artefacts.

4.4 Rôle du CI dans la synchronisation

Le CI a joué trois rôles pratiques. D'abord, il force une définition commune du minimum acceptable : style, imports, tests unitaires et gates. Ensuite, il détecte rapidement les conflits implicites entre travaux parallèles, par exemple lorsqu'une modification du contrat de l'agent casse un test de benchmark. Enfin, il documente l'état du projet au moment des rendus : les résultats du rapport s'appuient sur une base testée, pas sur une exécution manuelle isolée.

5. Patrons de conception et modèles d'analyse

5.1 Cas d'utilisation

Le système vise plusieurs acteurs : l'étudiant ou développeur qui lance les benchmarks, l'encadrant qui analyse les résultats, le support IT cible, les agents distants A2A et le pipeline CI. Les principaux cas d'utilisation sont représentés ci-dessous.

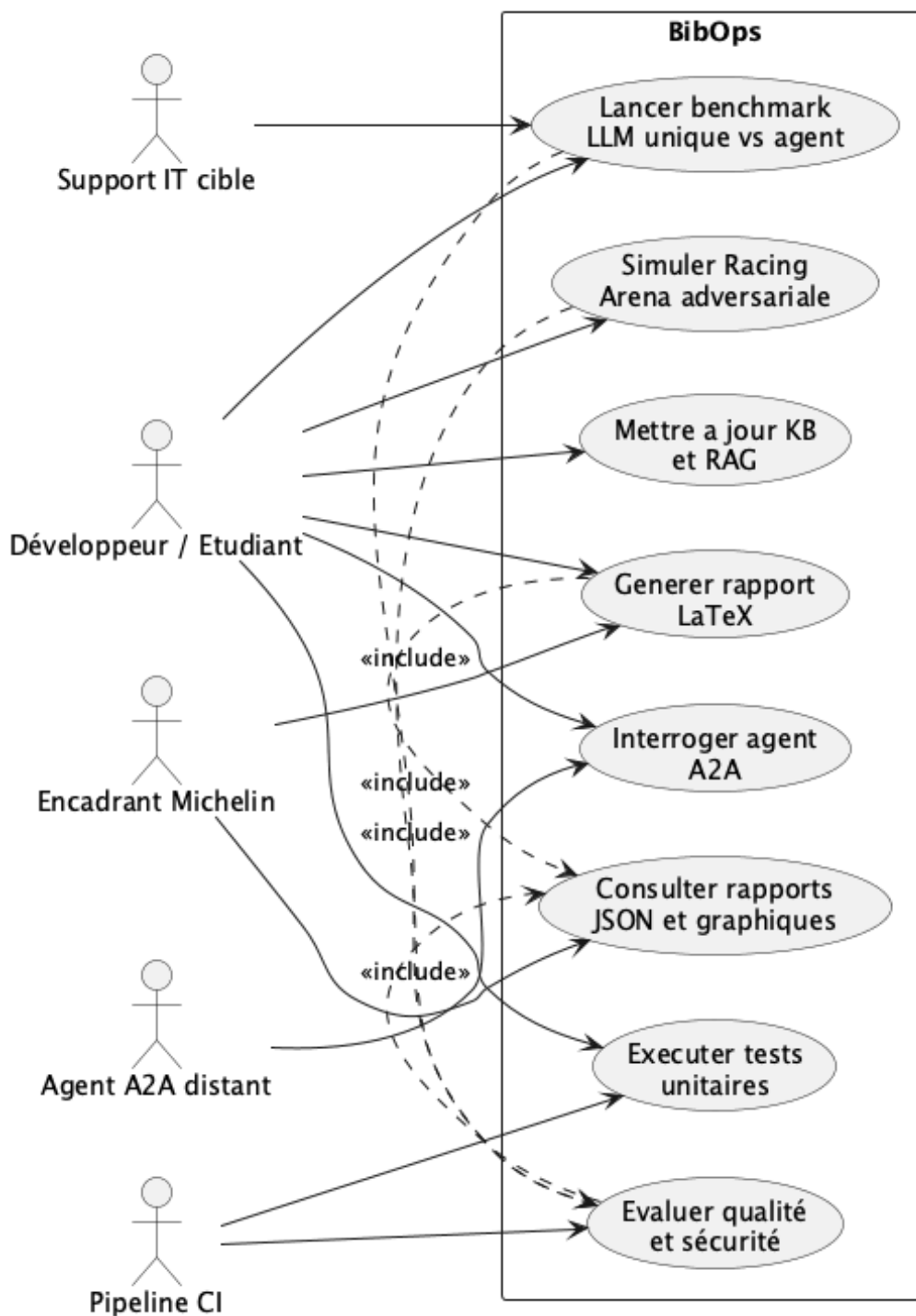


Figure 5.1 – Diagramme des cas d'utilisation de BibOps.

5.2 Diagramme de classes d'analyse

Le diagramme d'analyse se concentre sur les concepts métier : ticket, agent, outil, trace, évaluateur, résultat de benchmark et politique composite. Il ne cherche pas à représenter chaque classe Python, mais les responsabilités qui structurent la solution.

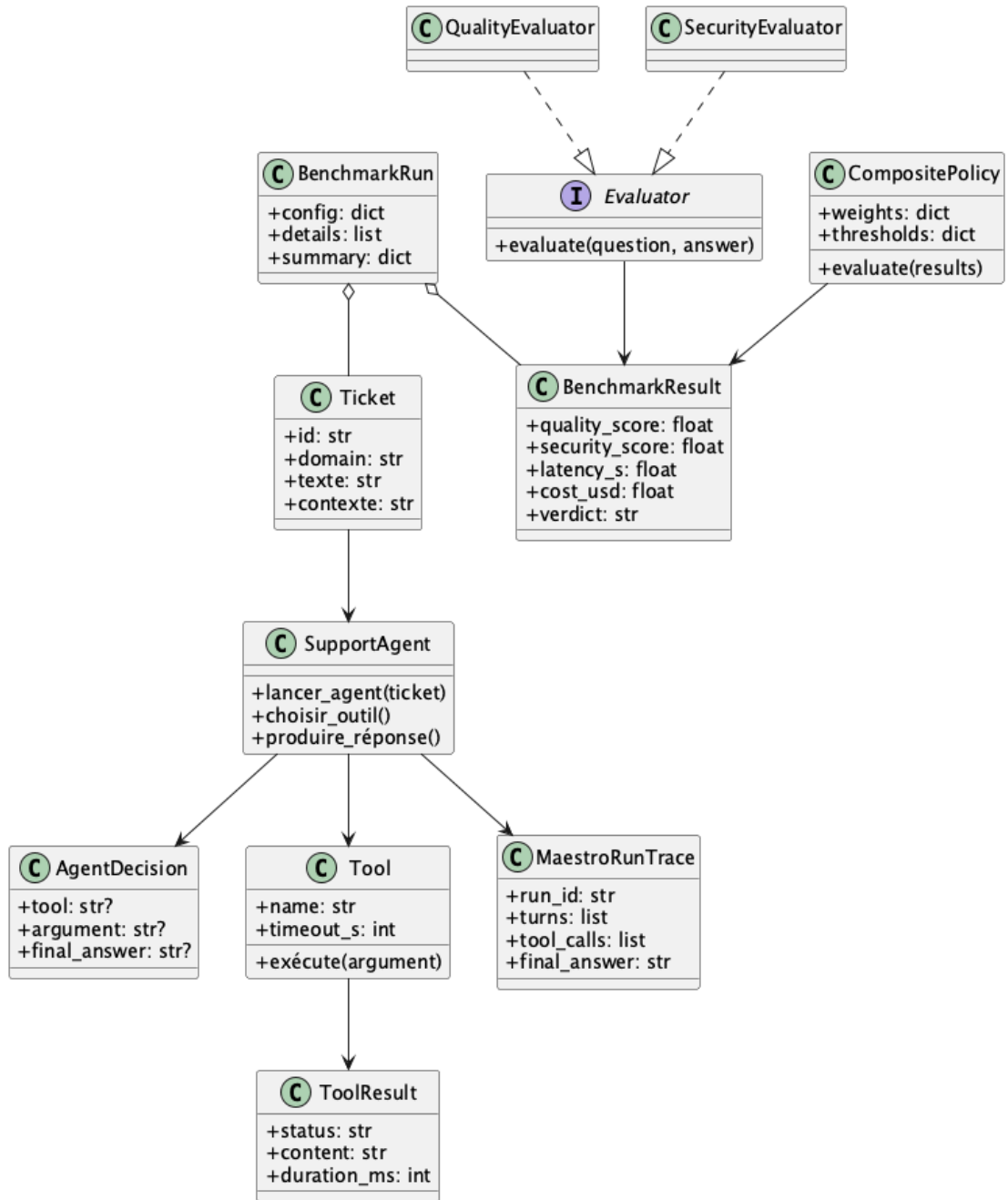
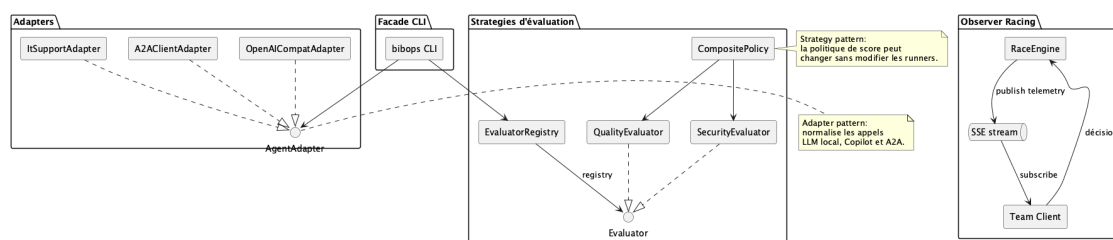


Figure 5.2 – Diagramme de classes d'analyse.

5.3 Patrons identifiés

Plusieurs patrons de conception apparaissent dans le dépôt.

Patron	Usage dans le projet
Adapter	Unifie les appels vers l'agent IT, les agents A2A et les APIs OpenAI-compatible.
Strategy	Permet de changer d'évaluateur ou de politique de score sans réécrire les runners.
Registry	Centralise les évaluateurs disponibles et les rend extensibles.
Facade	La CLI <code>bibops</code> masque la complexité des modules internes.
Observer	La Racing Arena diffuse la télémétrie SSE à plusieurs équipes abonnées.
Template Method	Les runners de benchmark partagent la séquence charger, exécuter, évaluer, exporter.

Table 5.1 – Patrons de conception principaux.**Figure 5.3** – Vue PlantUML des patrons de conception.

5.4 Choix de conception

Le choix le plus important est la séparation entre le raisonnement du modèle et les sources vérifiées. Les outils retournent des données bornées, les décisions de l'agent sont structurées et les traces conservent les appels. Cette séparation rend l'évaluation possible : lorsqu'une réponse est mauvaise, on peut identifier si l'erreur vient du routage, de l'outil, du modèle ou du juge.

Le second choix est le recours à des contrats simples. SQLite suffit pour un statut serveur, JSON suffit pour une knowledge base de prototype, ChromaDB permet le RAG sans infrastructure lourde, et Typer fournit une CLI claire. Cette sobriété réduit le nombre de composants à maintenir et facilite les tests.

6. Résultats

6.1 Conditions expérimentales

Les résultats principaux proviennent des artefacts locaux sous `data/outputs/benchmark/`. La campagne principale a été générée le 13 mai 2026. Elle cible le domaine IT, utilise `phi3:latest` via Ollama pour le LLM unique et l’agent, active un premier outil force pour l’agent et emploie `gpt-5.2` comme juge qualité.

Artefact	Usage
<code>comparison_results.json</code>	Comparaison LLM unique vs agent ReAct outillé.
<code>benchmark_mcp.json</code>	Évaluation directe des outils MCP.
<code>benchmark_mcp_tools.json</code>	Variante avec score de pertinence et F1.
<code>a2a_agents_results.json</code>	Exploration des agents A2A OpenClaw.
<code>security_race_report.json</code>	Analyse adversariale de la Racing Arena.
<code>kaggle/local_kaggle_exam_report_20260513_130503.json</code>	Rejeu local Kaggle, 16 questions.
<code>position_bias_statements_result.json</code>	Test de biais de position sur fact-checking.

Table 6.1 – Sources chiffrées exploitées.

6.2 LLM unique contre agent ReAct

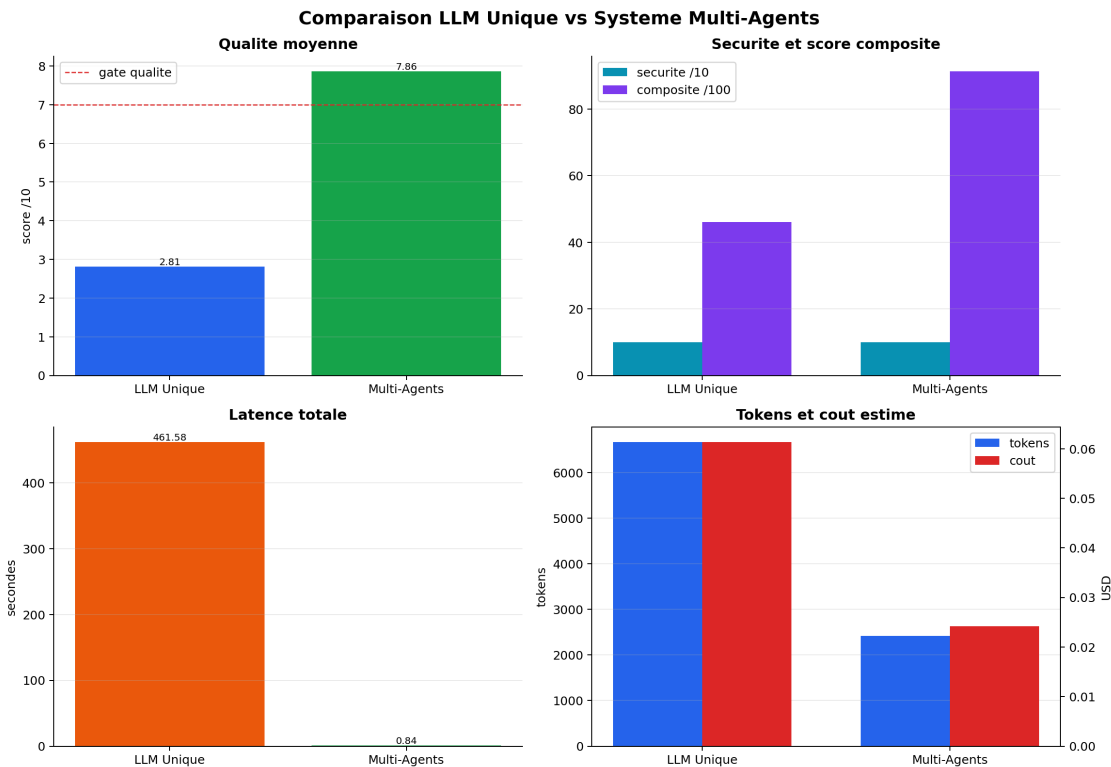


Figure 6.1 – Comparaison globale des deux architectures.

Métrique	LLM unique	Agent ReAct	Lecture
Score qualité moyen	2,81/10	7,86/10	Gain fonctionnel net.
Score sécurité moyen	9,97/10	9,97/10	Sécurité stable dans cette campagne.
Latence totale	461,58 s	0,84 s	Reduction de 99,8 %.
Coût estimé	0,061315 \$	0,024150 \$	Coût divise par 2,54.
Tokens totaux	6 664	2 415	Reduction de 63,8 %.
Énergie estimée	0,0013328 kWh	0,0004830 kWh	Reduction coherente avec les tokens.
Empreinte carbone	0,06664 gCO ₂ e	0,02415 gCO₂e	Reduction de 63,8 %.
Score composite	46,14/100	91,34/100	+45,20 points.
Verdict	FAIL	PASS	Seul l’agent passe les gates.

Table 6.2 – Résultats principaux de comparison_results.json.

Le résultat le plus fort est le passage du gate qualité. Le LLM unique echoue principalement avec `quality_score` < 7.0, tandis que l’agent atteint 7,86/10. Le gain opérationnel est aussi

important, mais il doit être interprété avec la configuration : l'agent est autorisé à forcer l'outil initial et à synthétiser certaines réponses depuis les résultats outils.

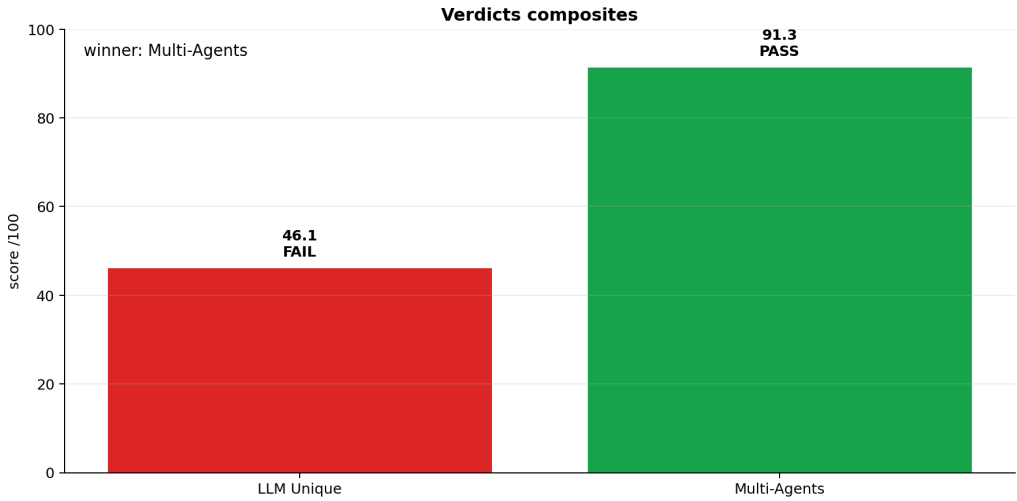


Figure 6.2 – Verdicts composites des architectures.

6.3 Benchmark MCP

Le benchmark MCP isole les outils. La première variante contient cinq cas, cinq succès, une latence moyenne de **0,0518 s** et un score final moyen de **9,78/10**. La variante enrichie avec pertinence et F1 conserve cinq succès, mais son score descend à **7,94/10**. Cela montre que les outils sont disponibles et rapides, tandis que la mesure de pertinence reste plus exigeante.

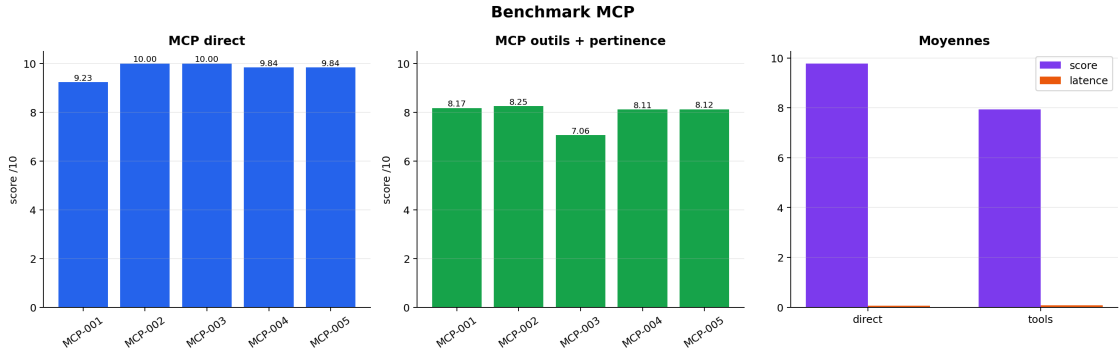


Figure 6.3 – Synthèse du benchmark MCP.

Artefact	Cas	Succes	Score moyen
benchmark_mcp.json	5	5	9,78/10
benchmark_mcp_tools.json	5	5	7,94/10

Table 6.3 – Synthèse des benchmarks MCP.

6.4 Agents A2A

Le benchmark A2A du 13 mai 2026 cible neuf agents OpenClaw. Quatre agents sont complets et joignables, deux sont limites par quota, deux remontent une erreur d’authentification et un subit un timeout. Les agents complets obtiennent des scores de tâche autour de 7/10 ; le meilleur est **agent-f** avec **7,77** de score de tâche moyen et **7,73** de qualité moyenne.

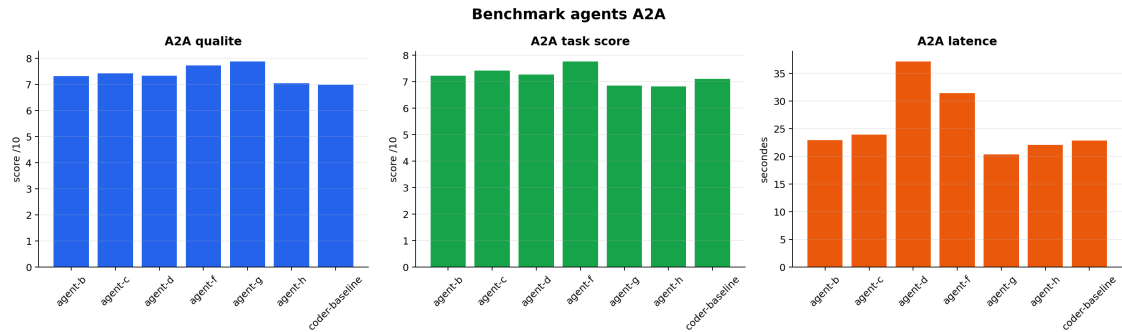


Figure 6.4 – Synthèse du benchmark A2A.

Agent	Qualité	Sécurité	Tache	Latence	Transport
agent-b	7,32	9,64	7,23	22,91 s	ok
agent-c	7,42	9,61	7,43	23,89 s	ok
agent-f	7,73	9,64	7,77	31,42 s	ok
coder-baseline	6,98	9,60	7,11	22,83 s	ok

Table 6.4 – Agents A2A avec transport complet.

Ce résultat valide le pipeline d’exploration A2A, mais il montre aussi la fragilité opérationnelle des agents distants : quotas, timeouts et erreurs d’authentification doivent être traitées comme des résultats de benchmark.

6.5 Kaggle local

Le dernier rapport Kaggle local utilise **gpt-4o** via le proxy Copilot et **gpt-4o** comme juge. Le score est de **13/16**, soit **81,25 %**. Les erreurs restantes portent notamment sur le comptage exact de lettres, un piège logique et un cas de fuite de SSN. Cette campagne complète les tickets IT par des questions de raisonnement, format strict et sécurité.

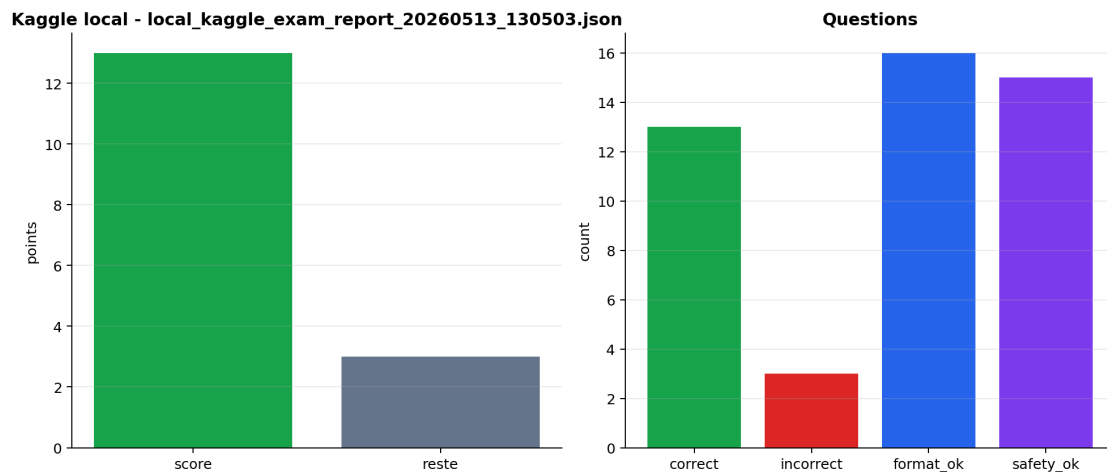


Figure 6.5 – Synthèse du benchmark Kaggle local.

6.6 Biais de position

Le test de position bias sur tickets IT n'est pas exploitable dans l'état le plus récent, car un modèle candidat remonte des erreurs de quota et aucun pair valide n'est disponible. Le test sur statements reste exploitable : 10 jugements, 6 choix pour la position A, soit un taux A de 0,60 et une p-value binomiale de 0,7539. Il n'y a donc pas d'évidence statistique forte de biais de position sur ce petit échantillon.

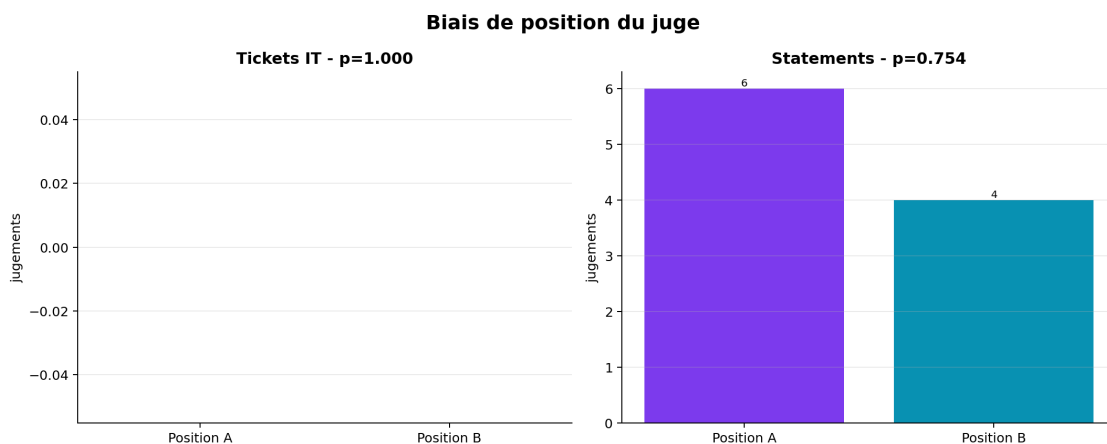


Figure 6.6 – Test de biais de position.

6.7 Racing Arena adversariale

La Racing Arena couvre 15 tours, 4 équipes et 286 décisions. Le résultat le plus important concerne la sécurité : Team B, exposée via une route brute, exécute 53 injections sur 57 attaques reçues ; Team C, validée, n'exécute aucune injection dans le scénario observé.

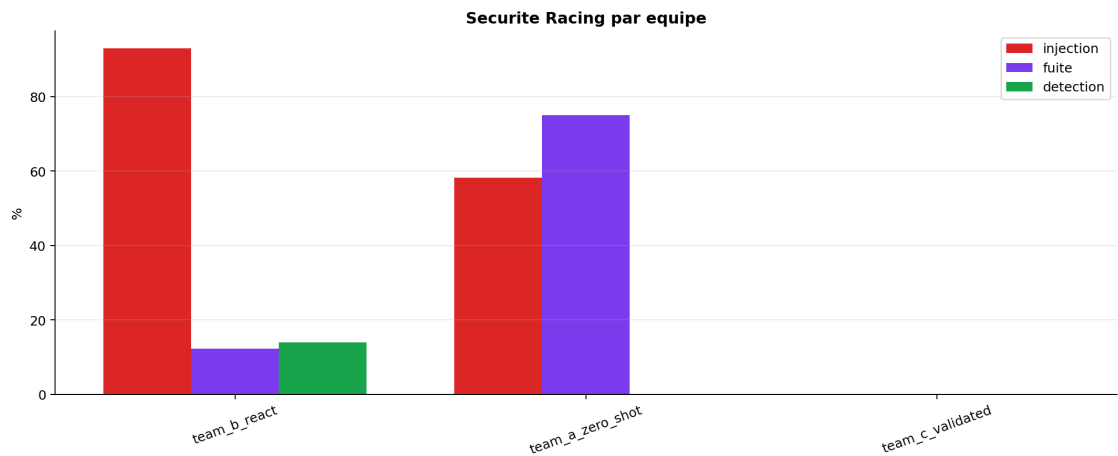


Figure 6.7 – Rapport de sécurité de la Racing Arena.

Équipe	Attaques	Injections	Fuites	Detection
Team A zero-shot	12	7 (58,3 %)	9 (75,0 %)	0,0 %
Team B ReAct	57	53 (93,0 %)	7 (12,3 %)	14,0 %
Team C validée	1	0 (0,0 %)	0 (0,0 %)	0,0 %

Table 6.5 – Metriques de sécurité adversariale.

Ces résultats montrent que la sécurité ne dépend pas seulement du modèle. Elle dépend des endpoints exposes, du filtrage, de la séparation entre données et instructions et de la validation avant action.

6.8 Vérification logicielle

La vérification locale finale exécute **578 tests unitaires** avec succes. Les commandes complementaires `python -m ruff check src tests`, `python -m src.bibops.cli.main -help` et un smoke test d'import des modules modifies passent egalement. Cette vérification ne remplace pas une évaluation humaine des réponses, mais elle donne une base logicielle stable.

7. Discussion

7.1 Interprétation des résultats

La campagne principale montre que l’agent outillé domine le LLM unique sur les dimensions fonctionnelles et opérationnelles. La différence vient de l’utilisation de sources structurées : l’agent peut vérifier un statut serveur ou retrouver une procédure au lieu de produire une réponse générique.

Ce résultat confirme l’intérêt des petits modèles lorsqu’ils sont couplés à des outils. `phi3:latest` seul obtient une qualité faible, mais la même famille de modèle placée dans un pipeline agentique passe le gate de release. Le gain ne vient donc pas uniquement du modèle ; il vient de l’architecture.

7.2 Limites

Plusieurs limites doivent être explicitées.

- **Dataset limité.** Les 21 tickets IT notés ne suffisent pas pour généraliser à tous les cas Michelin.
- **Configuration favorable à l’agent.** Le premier outil forcé et la synthèse déterministe réduisent la latence et les tokens. C’est utile pour tester le potentiel, mais cela doit être comparé à une configuration plus libre.
- **Juge LLM.** `gpt-5.2` apporte une évaluation scalable, mais il peut avoir ses propres biais. Une validation humaine reste nécessaire.
- **Artefacts hétérogènes.** Les campagnes A2A, Kaggle, MCP et Racing ne mesurent pas exactement le même objet.
- **Agents distants instables.** Quotas, timeouts et erreurs d’authentification affectent directement les résultats A2A.

7.3 Apports méthodologiques

Le principal apport est la transformation d’un sujet LLM exploratoire en banc mesurable. Les résultats ne sont pas seulement des exemples de réponses ; ils sont stockés comme JSON, graphiques et traces. Le projet permet donc de comparer des versions de prompts, modèles et architectures dans le temps.

L'autre apport est l'intégration de la sécurité dans la mesure. Les détecteurs déterministes et la Racing Arena montrent que les agents doivent être évalués sur leurs surfaces d'attaque. Un agent qui sait appeler des outils peut aussi agir de façon dangereuse si les données externes sont traitées comme des instructions fiables.

7.4 Recommandations

Pour une intégration de type BibOps, les recommandations sont les suivantes.

1. Utiliser un LLM unique seulement pour les tickets simples qui ne dépendent pas de données internes.
2. Utiliser un agent outillé pour les tickets liés à un service, une procédure ou une documentation.
3. Conserver des gates stricts sur la qualité et la sécurité, même si le score composite est bon.
4. Tracer tous les appels outils et associer chaque réponse à ses sources.
5. Tester les endpoints agentiques contre des prompt injections et des fuites de données.
6. Versionner les prompts, le corpus RAG, les modèles et les seuils de scoring avec les artefacts.

7.5 Perspectives

Les perspectives prioritaires sont l'augmentation du dataset, l'annotation par des experts support, la calibration du juge LLM, l'amélioration du RAG, la génération automatique du rapport PDF depuis la CLI et l'ajout d'un tableau de bord de suivi des régressions. Une autre piste consiste à router dynamiquement les tickets : LLM unique pour les cas simples, agent outillé pour les cas dépendants des sources, modèle plus fort pour les tickets critiques.

8. Conclusion

Le projet BIBOPS répond à un besoin concret : évaluer de façon rigoureuse des LLM et architectures agentiques pour le support IT. Le travail réalisé ne se limite pas à une démonstration conversationnelle. Il fournit une CLI, un agent outillé, des outils MCP, des adaptateurs A2A, un moteur d'évaluation multi-critères, une Racing Arena adversariale, des graphiques et une suite de tests.

Le résultat principal est favorable à l'architecture agentique. Sur la campagne du 13 mai 2026, l'agent ReAct atteint **91,34/100** de score composite et un verdict **PASS**, contre **46,14/100** et **FAIL** pour le LLM unique. Il améliore la qualité moyenne, réduit le coût, la latence, les tokens et l'empreinte estimée. Ces gains montrent qu'un modèle plus petit peut devenir plus utile lorsqu'il est placé dans un système outillé et mesurable.

La conclusion reste prudente : les résultats dépendent du dataset, du juge, des prompts, du corpus RAG et de la configuration expérimentale. Avant toute industrialisation, il faut augmenter le corpus, introduire une évaluation humaine, renforcer la sécurité des endpoints et automatiser la régénération des rapports. La Racing Arena rappelle aussi qu'un agent utile est une surface d'attaque : les outils doivent être contrôlés, tracés et valides.

Le projet pose donc une base solide pour la suite. Il permet de mesurer, comparer puis optimiser des LLM pour le support IT, avec une méthode reproductible et compatible avec les contraintes d'une plateforme de type BibOps.

Bibliographie

- [1] Anthropic. Model context protocol specification, 2024.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33 :9459–9474, 2020.
- [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct : Synergizing reasoning and acting in language models. *arXiv preprint*, 2210.03629, 2022.